

Mario Expert System

--design report

Boqing Yin (Peter) byin236

August 29, 2024

This report details the design and implementation of an expert system to play the classic video game Super Mario Land. The goal was to create an expert system that can make real-time decisions, reaching as far as possible in the game, and achieving high scores. The report demonstrates the methodology of the system's design, justifying critical choices in the expert system's selection and decision-making procedures.

A performance comparison is used to evaluate the system's effectiveness and highlight the strengths and weaknesses of the designed expert compared to humans and other machines. This analysis shows the system's capabilities and pointing out any potential improvements that can be made to this designed expert.

1. Design:

An expert system is generally anything that automates a process or method or controls something that is not modern machine learning. The expert system for Super Mario was designed with a rule-based architecture, where a set of predefined rules prompts the agent's actions. The knowledge base contains game-specific information, such as enemy location and level layouts, which the expert uses to make real-time decisions.

The design focused on effectiveness, ensuring the system could quickly respond to in-game challenges. Critical design choices, such as the prioritization of enemy engagement over obstacle avoidance and coin collections, were made to optimize the agent's success in navigating the game environment.

1.1 Choosing Expert System:

As lecture demonstrated to us, we have Rule-Based System, Case-Based System, Model-Based Reasoning System, Bayesian Networks, and Frame-Based System.

The first step in the development of super Mario expert system is to find a good representation.

	Rule-Based System	Case-Based System	Model-Based Reasoning System	Bayesian Networks	Frame-Based System
Pros	Simplicity: Easy to understand, implement, and modify. Deterministic: Provides clear and predictable outputs based on the rules.	Learning from Experience: Uses past cases to handle new situations, making it adaptable. Efficiency in Similar Situations: Quickly finds solutions for problems like those previously encountered.	Deep Understanding: Provides reasoning based on a model of how the system operates, allowing for insightful problem-solving. Predictive Power: Can predict outcomes and diagnose issues with precision.	Handling Uncertainty: Excellent at managing uncertainty and making decisions with incomplete information. Probabilistic Reasoning: Can update beliefs as new information becomes available.	Structured Knowledge: Organizes knowledge in a hierarchical manner, making it easy to manage and extend. Inheritance and Reusability: Supports inheritance, allowing for efficient reuse of knowledge.
Cons	Scalability Issues: Becomes complex and hard to manage with many rules. Rigidity: Not adaptive; struggles with situations not covered by	Dependency on Case Library: Requires a large and diverse set of cases to be effective. Limited Generalization: May not perform well in novel or significantly	Complexity: Requires a detailed, accurate model, which will be difficult to create and maintain. Resource-Intensive: Often computationally demanding and not suited	Complex Setup: Requires careful design and understanding of dependencies between variables. Computational Overhead: Will be resource-heavy,	Limited Reasoning Capability: Focuses more on organization than on complex decision-making. Rigidity in Representation: May

	predefined rules.	different scenarios.	for real-time applications like gaming.	especially in complex networks.	struggle with representing dynamic or unstructured information.
Suitability	It is suitable for choosing straightforward actions in Super Mario, such as jumping over an enemy or collecting a coin, but it could struggle with more complex or dynamic scenarios that require adaptive behaviors.	It is suitable for selecting actions in Super Mario when the current situation closely similar to the past scenarios, allowing the agent to learn from predefined experience, but it could be less effective in novel or unique situations that is not covered by its existing case library.	It could be suitable for choosing actions in Super Mario by predicting the outcomes based on a detailed designed model of the game's mechanics, but it is very likely to be very complex and computationally intensive for real-time gameplay and super Mario does require quick response.	It could be suitable for making decisions in Super Mario when dealing with uncertainty, such as estimating the probability of success for a risky jump, but the complexity of setting up and processing these networks will affect real-time performance. Also, the game complexity is considerably low and there would have no needs for such measures.	It could be suitable for organizing knowledge about different game elements in Super Mario, such as capture enemies and obstacles, but it is limited in its ability to directly choose complex or dynamic actions during gameplay.

Above are the expert system's pros and cons for all the representations that we have learnt in COMPSYS 726.

After comparing the advantages and disadvantages of the expert system. The choice to implement a rule-based expert system is chosen. The Super Mario agent was primarily motivated by its structured and transparent decision-making capabilities, as well as its efficiency in resource usage. Rule-based systems, by their nature, require minimal computational power compared to more complex expert systems like Bayesian networks or Model-Based Reasoning Systems, making them highly suitable for applications where low response time is critical. This efficiency is particularly beneficial in gaming environments like Super Mario, where real-time responsiveness is essential. By utilizing predefined rules that directly map to specific in-game actions, the system can quickly process and execute decisions without the need for extensive computational resources. The deterministic nature of the rule-based expert system was heavily considered to improve the system's robustness and make it easier to debug. This not only ensures smooth operation in the game but also allows for the system to be easily scaled or adapted to other scenarios with minimal overhead. Theoretically, the designed Rule-Based Mario expert can pass any level if the rules cover all possible scenarios.

1.2 Expert Implementation:

People who have worked with a rule-based expert system know that "If Else" is stacked together to provide comprehensive support for that expert to use in actual operations. The input to the expert system is state, game_area, and frame. Memory access reading does add more inconsistency to the expert's opinion; therefore, it is not chosen for this implementation. Also, limiting the expert to only having access to the latest frame at a time from game_area will make it faster to process, easier to understand, and debug. Although the resolution of the game area array is relatively lower than the direct memory access, which can result in loss of accuracy, and due to the frame not updating as often as the memory and containing less information, some parameters of Mario will not be received such as velocity and the more precision location.

I am starting with a straightforward rule for my system that is always going right. Super Mario is a simple game, and the ultimate final goal is to reach the rightmost destination. After reaching the first monster, I added the detection method to detect the monster approaching Mario. If a monster can be defeated by approaching Mario, it will hold its position and jump when it is close enough for Mario to jump on top of that monster and step on it to achieve a kill. With the existence of the final rule, I can monitor my expert's behaviors and implement new rules when the existing rules are insufficient.

The initially designed system struggled when the monsters approached from above Mario or if multiple monsters were approaching with a small distance in between. To resolve this issue, new rules are introduced with less priority. After the new rules, Mario can skip monsters that are hard to kill and wait on any monster that enters the field but is not at the same height as Mario since all monsters in the first stage are moving left.

After reaching the plate floating in the air, I noticed a coin behind the Mario after it landed on the ground level. The expert could already claim the coin and mushroom, but it only works when they are in front of the Mario. At this point, I

implemented some search functions that can be used to search for target values from the scope we want. `cordBehind`, `cordBelow`, and `cordAbove` functions are coded and introduced to the expert system to help search for target values that the expert system is looking for. If a target value is found, a different set of rules will be activated to drive Mario differently from the expected behaviors.

Mario will encounter different terrains, and similar but slightly different rules will be applied to each different terrain, making it perform better for that specific terrain. Different monsters entering the field will also denote different rules the expert system will take. When designing all the rules above, I monitor the expert instructions and parameters along with the gameplay and play the game myself to see what action I will take in the same place.

When controlling the system, each time the `choose_action` function is called an action, it will result from the implemented rules and be inputted into the game. The expert acts on only one button for each function call. The expert only does one action at a time, such as moving left/right or jumping, before the following function call. This approach is straightforward and easier to understand and implement. This restriction simplifies the expert's logic, making it easier to understand and debug. This enhanced predictability facilitates issue correction during development. When debugging, tracking, and analyzing the expert's behavior becomes more straightforward, as each action is confined to a single call, minimizing the chances of an action that is hard to trace.

2. Justification:

In this section, we will provide a thorough justification for the design and development of the Mario expert system. This includes a detailed evaluation of the designed system, comparing its performance against alternative approaches or speed runners, identifying any limitations encountered during development, and suggesting potential improvements.

2.1 Evaluating the Designed system:

The game itself does have some figures that can be used to compare the performance of the designed expert system, such as life, score, and time used at the end of each stage. As we are measuring the performance of the gameplay, we can also add counters to the system and count the frequency of rules being used, and operations that are accomplished.

To find some statistics that will be useful in measuring the performance of the game I have chosen the following:

Time Taken: This measurement records the time the expert system takes to complete a level. It is a critical statistic of the system's efficiency in decision-making and execution. Faster completion times suggest more effective navigation and obstacle avoidance, while longer times may indicate delay or inefficiencies in rule inference.

Score: This measurement captures the score accumulated by the system at the stage 1 checkpoint of that level. The score reflects the agent's ability to perform actions that are rewarded in the game, such as collecting coins, defeating enemies, and other collectables. High scores at checkpoints demonstrate the system's proficiency in collecting in-game rewards.

Number of deaths: The total number of deaths is a key indicator of the system's robustness and error rate. A lower number of deaths implies that the expert system is effective at avoiding fatal mistakes and successfully navigating through the map. This statistic helps assess the reliability of the system's rule-based decision-making process.

Monster Killed: This statistic counts the number of enemies defeated by the agent during gameplay. It measures the system's aggressiveness and its ability to successfully engage and neutralize threats. A higher count of monsters killed suggests a more proactive approach, while lower numbers may indicate a more cautious or efficient first strategy.

Coins collection: The number of coins collected is an additional statistic that reflects the system's integrity and exploration capabilities. Collecting more coins would require the agent to explore the level more comprehensively and make strategic decisions about when and where to go off from the shortest path. This statistic provides an overview of the system's ability to balance exploration with efficient level completion.

The evaluation statistics chosen—Time Taken, Score at Checkpoint, Number of Deaths, Monsters Killed, and Coins Collected—provide a detailed assessment of the expert system when playing Super Mario Land. Those statistics are essential in quantifying the performance of the expert system. The statistics not only validate the effectiveness of the designed system but also highlight specific aspects that can be improved in future design to improve the expert system's overall performance.

2.2 Performance Comparison:

The result of the designed expert system is compared against to 3 different players, general human which is the designer who first time play the Super Mario Land, top speed runners are the people who post their result on the speedrun leaderboard, and the machine player is another accomplish of machine playing Super Mario Land. Since the designed expert system cannot complete stage 2 only stage 1's statistics is being compared.

Measures	Designed Expert	General Human Player	Top Speed runner	Another Machine player
Time Taken(s)	103.03	124.52	38.08	80.00
Score	8810	6130	4610	5360
Number of Deaths	1	2	0	0
Monster Killed	12	5	1	4
Coin collection	21	18	9	5

The performance comparison of the designed expert system against a general human player, top-level speed runners, and another machine player reveals a difference in its capabilities. The expert system outperformed the general human player in terms of time taken, score, number of deaths, monsters killed, and coins collected, demonstrating its efficiency and effectiveness in the game. However, it falls short when compared to the top speed runners, particularly falling behind significantly on time taken. The expert system also shows competitive performance against another machine player, especially in score, monster kills, and coin collection, although it still takes longer to complete the stage. These results highlight the strengths of the designed expert system, especially in achieving a balanced strategy, while also pointing out the areas where further optimization of the system can make it closer to top human players and other machines.

2.3 Limitations:

The expert system's time to complete Stage 1 is significantly longer than that of top-level speed runners and another machine player, indicating the decision-making process can be optimized to a more time-first strategy. Also, the expert does not collect or kill all monsters in the first stage, which could mean the search procedure of the expert is not comprehensive enough, for the collection and score first strategy this does indicates a huge improvement can be made.

2.4 Available improvements:

In stage 2, with the new monster Bee introduced to the game, Bees behave and attack differently from any monster that has been seen in stage 1. This indicates the current expert system does not have an existing rule that is good to face such a monster, and new rules should be designed for Bees.

The resolution of the game_area is limited; it would be better for action precision if the game information could be read from the memory. Memory would have data with better resolution and additional details that cannot be received from reading the screen representation of the game. This would help make more precious and more confident moves. The action is currently limited to one specific action per call; if multiple actions can be performed during the same call, the flexibility of the expert system can be improved.

References:

1. J. Togelius, S. Karakovskiy, J. Koutn'ík, and J. Schmidhuber, "Super mario evolution," in CIG'09: Proceedings of the 5th international conference on Computational Intelligence and Games. Piscataway, NJ, USA: IEEE Press, 2009, pp. 156–161.
2. F. Dignum, J. Westra, W. van Doesburg, and M. Harbers, "Gamesand agents: Designing intelligent gameplay," International Journal of Computer Games Technology, vol. 2009, 2009.
3. Cig2009competition. [Online]. Available: <http://julian.togelius.com/mariocompetition2009/CIG2009Competition.pdf>
4. Karadayi-Usta, S. (2024). Chapter 35 - Fuzzy rule-based systems: How to construct a FRBS with MATLAB, R, and Python. In T. Allahviranloo, W. Pedrycz, & A. Seyyedabbasi (Eds.), Decision-Making Models (pp. 623–643). Academic Press. <https://doi.org/10.1016/B978-0-443-16147-6.00008-6>
5. Sydenham, P. H., & Thorn, R. (Eds.). (2005). Handbook of measuring system design. Wiley.
6. Any% in 12:21.200 by Shaaf—Super Mario Land—Speedrun.com. (n.d.). Retrieved August 30, 2024, from <https://www.speedrun.com/sml1/runs/y4k8k0dm>